

STUDENT PERSONAL FINANCE MANAGER

Python and MySQL Based Desktop Application

1. Project Title

Student Personal Finance Manager

A Python and MySQL-based desktop application designed to help college students record, manage, analyze, and monitor their personal expenses, monthly budgets, and savings goals.

2. Problem Identification

College students often have a limited monthly allowance or fixed amount of money available for their daily needs. However, many students do not maintain a proper record of their expenses.

As a result, students may not know:

- How much they have spent
- Where most of their money is going
- How much money is remaining
- Whether they are approaching their monthly budget
- How much they can save
- Whether they are progressing toward a savings goal
- How their spending changes over time

A simple expense tracker can record transactions, but it may not provide useful analysis of spending patterns.

Therefore, there is a need for a simple and user-friendly system that can combine **expense tracking, budget management, spending analysis, savings tracking, and basic financial insights** in one application.

3. Problem Statement

To develop a Python-based Student Personal Finance Manager that enables college students to record and manage daily expenses, set and monitor monthly budgets, analyze spending patterns, track savings goals, receive rule-based alerts and recommendations, and view financial information through a simple graphical interface backed by a MySQL database.

4. Proposed Solution

The proposed system will be a desktop application developed using **Python, Tkinter, MySQL, and basic Python libraries**.

The application will provide a graphical interface through which students can manage their personal finances.

Python will handle:

- Application logic
- Calculations
- Input validation
- Data processing
- Spending analysis
- Budget alerts
- Rule-based recommendations

Tkinter will provide the graphical user interface.

MySQL will provide permanent storage for student information, expenses, budgets, and savings goals.

Basic Python libraries such as `datetime`, `csv`, `statistics`, and `matplotlib` may be used where appropriate.

5. Main Objectives

The project aims to:

1. Provide a simple system for recording daily expenses.
2. Allow students to set and manage a monthly budget.
3. Calculate total spending and remaining budget.
4. Categorize and analyze expenses.
5. Identify the highest spending categories.
6. Provide daily, weekly, and monthly spending summaries.
7. Allow students to create and monitor savings goals.
8. Generate basic budget alerts.
9. Provide rule-based spending insights and recommendations.
10. Provide graphical representations of spending.

11. Allow users to search and filter expense records.
 12. Allow users to export expense information.
 13. Store financial records using MySQL.
 14. Demonstrate practical Python programming and computational thinking concepts.
-

6. Scope of the Project

6.1 Included Features

The main system will include:

- Student profile
- Expense management
- Monthly budget management
- Dashboard
- Daily/weekly/monthly analysis
- Category-wise spending analysis
- Savings goals
- Budget alerts
- Rule-based spending insights
- Charts and visualization
- Search and filtering
- CSV export
- MySQL database
- Tkinter graphical interface

6.2 Optional Features

If sufficient development time is available, the following may be added:

- Basic login/authentication
- Recurring expense tracking

These features will not be essential to the completion of the project.

6.3 Outside the Scope

The project will not include:

- Real bank account integration

- Online payment processing
- Banking APIs
- Investment management
- Cryptocurrency tracking
- AI-based financial advice
- Machine learning
- Real-time financial market data
- Cloud-based financial services

This keeps the project technically manageable and suitable for a first-year programming course.

7. Major Features

7.1 Student Profile

The user can create a basic student profile containing:

- Student ID
- Name
- Monthly budget

The information will be stored in MySQL.

7.2 Expense Management

The user can add an expense using:

- Date
- Category
- Amount
- Description

Example:

Category: Food
Amount: ₹250
Date: 28-09-2026
Description: Lunch

The system will provide options to:

- Add expense

- View expense
- Edit expense
- Delete expense

7.3 Expense Categories

Expenses can be organized into categories such as:

- Food
- Transport
- Education
- Entertainment
- Shopping
- Hostel
- Other

The categories can be used for further analysis.

7.4 Monthly Budget Management

The student can set a monthly budget.

Example:

Monthly Budget: ₹10,000

The system will calculate:

Total Spent:	₹6,850
Remaining:	₹3,150
Budget Used:	68.5%

7.5 Dashboard

The dashboard will provide a quick overview.

Example:

STUDENT PERSONAL FINANCE MANAGER

Monthly Budget:	₹10,000
Total Spent:	₹6,850
Remaining:	₹3,150
Budget Used:	68.5%

Highest Category: Food

[Add Expense]
[View Expenses]
[Budget Analysis]
[Savings Goals]

The dashboard will act as the main screen of the application.

7.6 Daily, Weekly and Monthly Analysis

The application will allow users to analyze spending over different periods.

For example:

Today's Spending: ₹350
This Week: ₹1,850
This Month: ₹6,850

This can help students understand their spending patterns.

The feature can be implemented using dates stored in the database and Python's `datetime` functionality.

7.7 Category-Wise Spending Analysis

The system will calculate spending for different categories.

Example:

Food	₹2,500
Transport	₹1,200
Education	₹1,500
Entertainment	₹1,000
Other	₹650

The system can identify:

Highest Spending Category: Food

It can also calculate the percentage contribution of each category to total spending.

7.8 Savings Goals

Students can create savings goals.

Example:

Goal Name: New Headphones
Target Amount: ₹5,000
Current Savings: ₹2,000

The system calculates:

Remaining: ₹3,000
Progress: 40%

The application can display progress visually.

Multiple savings goals may be supported.

7.9 Budget Alerts

The application will use simple rule-based conditions.

For example:

Budget Used < 80%

↓

Within normal range

Budget Used ≥ 80%

↓

Warning: Approaching budget limit

Budget Used ≥ 100%

↓

Alert: Budget exceeded

This will be implemented using Python conditional statements.

7.10 Rule-Based Spending Insights

The application will provide simple recommendations based on spending data.

For example:

Food represents 36% of your total spending.

Insight:

Food is currently your highest spending category.

Recommendation:

Consider setting a separate food budget.

These recommendations will use predefined rules.

They will **not use artificial intelligence or machine learning**, keeping the implementation simple and transparent.

7.11 Spending Charts

The application may use the `matplotlib` library to generate charts.

Possible charts include:

- Category-wise spending
- Weekly spending
- Monthly spending
- Budget usage

For example:



Charts will provide a visual representation of financial information.

7.12 Search and Filter

As the number of expense records increases, users should be able to find specific records.

Possible filters include:

- Category
- Date
- Amount
- Month

Example:

Search Category: Food

The system will display only relevant food expenses.

7.13 CSV Export

The user can export expense records into a CSV file.

Example:

expense_report.csv

The exported file may contain:

```
Date,Category,Amount,Description
28-09-2026,Food,250,Lunch
28-09-2026,Transport,80,Metro
29-09-2026,Education,500,Book
```

This feature can be implemented using Python's built-in `csv` library.

7.14 Optional Login

If time permits, a simple login system may be implemented.

Example:

```
-----  
STUDENT LOGIN  
-----  
  
Username: [ _____ ]  
Password: [ _____ ]  
  
[ LOGIN ]  
-----
```

This feature will be optional because it introduces additional authentication logic.

7.15 Optional Recurring Expenses

The application may allow students to mark expenses as recurring.

Examples:

- Monthly subscription
- Hostel expense
- Phone recharge
- Regular transportation

Example:

```
Expense: Phone Recharge  
Amount: ₹299  
Frequency: Monthly  
Recurring: Yes
```

This will be considered an optional extension rather than a core requirement.

8. Technology Stack

Programming Language

Python

Used for:

- Application logic
- Calculations
- Validation

- Data processing
- Analysis
- Rule-based recommendations

Frontend / GUI

Tkinter

Used to create:

- Dashboard
- Forms
- Buttons
- Tables
- Input fields
- Navigation screens
- Alerts

Database

MySQL

Used for permanent storage of:

- Student information
- Expense records
- Budget information
- Savings goals

Libraries

The project may use:

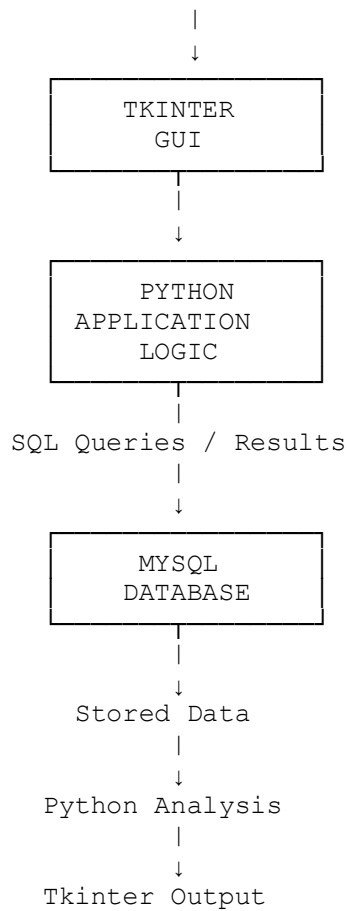
```
mysql-connector-python  
datetime  
csv  
statistics  
matplotlib
```

Only necessary libraries will be used.

9. Working Model

The complete system will work according to the following model:

USER



Simple explanation:

Tkinter = What the user sees

Python = The brain of the application

MySQL = The memory of the application

10. Complete Application Workflow

START



Open Application



Display Dashboard



Select Operation



Add Expense
Set Budget
View Expenses
Analyze Spending
Manage Savings Goal
Search / Filter
Export Report



```
Validate Input
↓
Store / Retrieve Data from MySQL
↓
Perform Calculations
↓
Analyze Spending
↓
Generate Alerts / Insights
↓
Display Results
↓
Return to Dashboard
↓
EXIT
```

11. Example Working Scenario

Suppose a student has:

Monthly Budget = ₹10,000

The student enters:

Food	₹2,500
Transport	₹1,200
Education	₹1,500
Entertainment	₹1,000
Other	₹650

The application calculates:

Total Spending = ₹6,850

Remaining Budget = ₹3,150

Budget Used = 68.5%

The system identifies:

Highest Spending Category = Food

The dashboard displays:

MONTHLY SUMMARY

Budget:	₹10,000
Total Spent:	₹6,850
Remaining:	₹3,150
Budget Used:	68.5%

Highest Category:
Food

Status:
WITHIN BUDGET

The system can then provide:

Insight:

Food is your highest spending category.

Recommendation:

Consider monitoring food expenses.

12. Computational Thinking Approach

The project demonstrates the four major aspects of computational thinking.

12.1 Decomposition

The overall problem is divided into smaller modules:

Expense Management
Budget Management
Spending Analysis
Savings Management
Alerts
Reports

12.2 Pattern Recognition

The system identifies patterns such as:

- Highest spending category
- Increasing weekly expenses
- High category spending
- Budget approaching its limit

12.3 Abstraction

Only relevant information is stored and processed.

An expense can be represented using:

Date
Category
Amount
Description

12.4 Algorithm Design

Algorithms are created for:

- Calculating total spending
- Calculating remaining budget
- Calculating budget percentage
- Finding the highest spending category

- Calculating savings progress
 - Generating budget alerts
 - Filtering expense records
-

13. Python Concepts Demonstrated

The project will demonstrate:

Basic concepts

- Variables
- Data types
- Operators
- Input/output

Decision making

- `if`
- `elif`
- `else`

Loops

- `for`
- `while`

Data structures

- Lists
- Tuples
- Dictionaries

Functions

Possible functions include:

```
add_expense()
view_expenses()
edit_expense()
delete_expense()
calculate_total()
calculate_remaining()
analyze_expenses()
set_budget()
create_savings_goal()
check_budget()
generate_insight()
```

Exception Handling

Used to handle:

- Invalid amounts
- Empty fields
- Incorrect input
- Database errors

File Handling

Used for CSV report generation.

Database Connectivity

Python will communicate with MySQL using SQL queries.

GUI Programming

Tkinter will be used to create the application interface.

14. Proposed Database Structure

The initial database can contain the following tables.

Students

```
student_id  
name  
monthly_budget
```

Expenses

```
expense_id  
student_id  
date  
category  
amount  
description
```

Savings Goals

```
goal_id  
student_id  
goal_name  
target_amount  
current_amount
```

The exact database structure can be finalized during implementation.

15. Development Plan

Phase 1 — Planning

- Finalize project problem
- Define objectives
- Define scope
- Design system architecture
- Design database

Phase 2 — Python Logic

- Create functions
- Implement expense calculations
- Implement budget calculations
- Implement savings calculations
- Implement validation

Phase 3 — MySQL

- Create database
- Create tables
- Connect Python to MySQL
- Implement CRUD operations

Phase 4 — Tkinter GUI

Build:

- Dashboard
- Add Expense
- Expense History
- Budget screen
- Analysis screen
- Savings Goal screen

Phase 5 — Additional Features

Add:

- Alerts

- Rule-based insights
- Charts
- Search/filter
- CSV export

Phase 6 — Testing

Test:

- Valid inputs
- Invalid inputs
- Empty fields
- Negative amounts
- Budget limits
- Savings calculations
- Search/filter
- Database operations

Phase 7 — Documentation

Prepare:

- Problem statement
- Objectives
- Scope
- Algorithm
- Flowchart
- Implementation details
- Source code
- Screenshots
- Sample input/output
- Conclusion

Phase 8 — Presentation and Viva

Each member will understand:

- Complete project workflow
- Python concepts

- Database structure
 - GUI
 - Algorithms
 - Their individual contribution
-

16. GitHub Repository Structure

The repository can be organized as:

```
student-personal-finance-manager/  
├── README.md  
├── src/  
│   ├── main.py  
│   ├── database.py  
│   ├── expenses.py  
│   ├── budget.py  
│   ├── savings.py  
│   └── analysis.py  
├── database/  
│   └── database.sql  
├── screenshots/  
├── documentation/  
│   └── project_report.pdf  
├── requirements.txt  
└── .gitignore
```

The repository will contain accessible source code, database scripts, documentation, screenshots, and project information.

17. Expected Outcome

The expected outcome is a functional desktop application that allows college students to:

- Record expenses
- Manage a monthly budget
- Analyze spending
- Monitor savings goals
- Receive budget alerts
- View spending insights
- Search and filter records

- Generate basic charts
- Export expense reports

The application will demonstrate practical use of Python, MySQL, Tkinter, basic Python libraries, database management, computational thinking, and problem-solving.

18. Future Scope

Possible future extensions include:

- Advanced authentication
- Mobile application
- Cloud database
- Automatic recurring transactions
- Advanced financial analytics
- AI-based financial assistance
- Personalized financial recommendations
- Multi-device synchronization

These features are outside the current project scope.

19. Conclusion

The **Student Personal Finance Manager** is designed to solve a practical problem faced by college students while providing a suitable platform for demonstrating programming and computational-thinking skills.

The project combines:

Python + Tkinter + MySQL + Basic Python Libraries

to create a complete desktop application.

The project maintains a balance between **functionality, simplicity, and educational value**. It contains enough features to demonstrate meaningful problem-solving while avoiding unnecessary technologies that could make implementation and individual viva difficult.

The core system will focus on expense management, budget monitoring, spending analysis, savings goals, alerts, and rule-based insights. Additional features such as charts, filtering, and CSV export can be implemented as extensions depending on the team's progress.

Most importantly, every group member will be expected to understand the complete system and clearly explain their own contribution during the individual viva voce.